

AI Email Reading Agent

TQTech | Software Engineer 2 — AI Automation Engineer

Core Idea

A real email account receives emails. The AI reads each incoming email, decides whether it is important or not, and if important, displays it as a notification on a dashboard. Non-important emails are silently ignored.

How It Works — End to End

1. An email arrives in the connected inbox (Gmail, IMAP, or mock data)
2. The AI reads the subject and body of the email
3. The AI decides: is this email important?
4. If YES — the email appears on the dashboard as a notification
5. If NO — the email is ignored, nothing is shown
6. The same email is never shown more than once

AI Classification — The Heart of the System

The AI must read and reason about each email. It must produce a structured decision for every email:

- important — true or false
- priority — HIGH, MEDIUM, or LOW
- category — e.g. PAYMENT_ISSUE, SERVER_DOWN, SPAM
- reason — a clear sentence justifying the decision

Accepted AI methods:

- OpenAI / Claude / Gemini API
- Local LLM via Ollama
- Rule-based classifier (minimum acceptable)
- Hybrid: rules + AI

Dashboard — Notification Display

When the AI marks an email as important, it must appear on the dashboard. The dashboard is where the user sees all important emails at a glance.

Each notification on the dashboard must show:

- Sender (From)
- Subject
- Priority (HIGH / MEDIUM / LOW)
- Category (e.g. PAYMENT_ISSUE)
- Reason the AI flagged it as important
- Time received

What the AI Should Flag as Important

- Client complaint or urgent customer request
- Payment failure or billing issue
- Low-priority automated or subscription email

Supporting Requirements

Email Source

- Connect to Gmail, IMAP, or use mock JSON/CSV data
- Mock mode is mandatory so the project can be tested without real credentials

Duplicate Prevention

- Once an email is processed and shown, never show it again
- Store processed email IDs in SQLite, JSON, PostgreSQL, or Redis

Scheduling

- Continuously poll the inbox on a set interval (e.g. every 2 minutes)

Docker

- The full project (agent + dashboard) must run with: docker compose up --build

Credentials

- Use your own email account or mock data — TQTech provides nothing
- Never push real API keys, tokens, or passwords to GitHub

How to Submit

1. GitHub repository with full source code
2. Live demo link showing the dashboard
3. Dockerfile + docker-compose.yml
4. .env.example with all variables (no real secrets)
5. Mock email dataset
6. README — setup, how AI works, how dashboard works, limitations
7. Submit the task to this google form: <https://forms.gle/uqkDo9kN6hfF4qj86>

Marking (100 Marks)

- AI importance detection logic — 40
- Dashboard notification display — 10
- Email reading / mock data — 20
- Duplicate prevention — 10
- Docker setup — 10
- README & documentation — 5
- Code quality — 5

Disqualification

1. No GitHub repo or README
2. Cannot explain your own implementation

Submission Deadline: June 13, 2026 (11:59 PM).

Note: We encourage submission even though you can not able to complete full project.